

```
1 pip install opencv-python matplotlib numpy
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cyclers>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
```

```
1 import cv2 # Import OpenCV library for image processing
2 import matplotlib.pyplot as plt # Import Matplotlib for displaying images
3
4 # Read the image from the disk into a NumPy array
5 img = cv2.imread('number_plate.jpg') # cv2.imread reads the image in BGR (Blue-Green-Red) format
6 # Convert the BGR image to RGB format since Matplotlib expects RGB
7 img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # This ensures colors look correct when displayed
8
9 # Display the image using Matplotlib
10 plt.imshow(img_rgb) # Render the RGB image array
11 plt.title('Original Image') # Add a title to the plot window
12 plt.axis('off') # Hide the X and Y axes (ticks and borders) for a clean look
13 plt.show() # Show the actual window on screen
14
15 # Print the dimensions of the loaded image
16 print("Image Shape:", img.shape) # Outputs (height, width, channels) e.g., (100, 400, 3)
```

Original Image

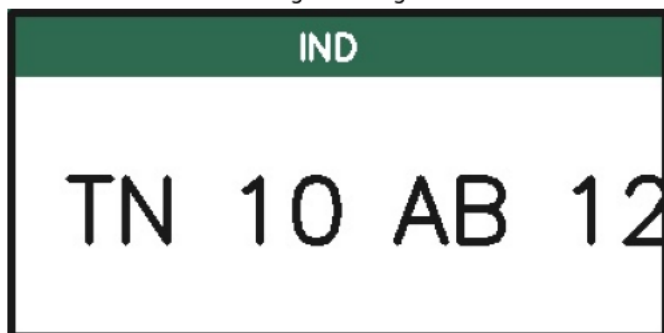
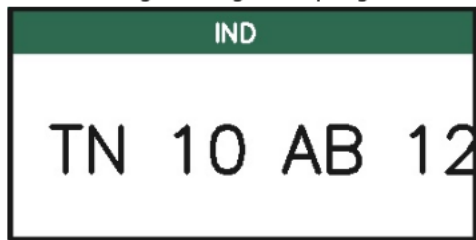


Image Shape: (200, 400, 3)

```
1 # Downsample the image to a tiny 50x25 resolution to simulate a very low sampling rate (bad camera sensor)
2 low_res = cv2.resize(img, (50, 25), interpolation=cv2.INTER_NEAREST) # INTER_NEAREST simply drops pixels without blending
3 print("Low-Res Shape:", low_res.shape) # The image is now only 25 pixels high and 50 pixels wide
4
5 # Scale the tiny image back up to 400x200 so we can actually see the blocky pixelation effect on screen
6 pixelated = cv2.resize(low_res, (400, 200), interpolation=cv2.INTER_NEAREST) # Upscale using Nearest Neighbor to keep it sharp
7
8 # Create a figure with a specific size (width=10, height=4 inches)
9 plt.figure(figsize=(10, 4)) # Create a visualization window to hold our images
10
11 # Create the left subplot (1 row, 2 columns, 1st position)
12 plt.subplot(1, 2, 1) # Position this image within a grid layout
13 plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # Display the original, high-quality image
14 plt.title('Original (High Sampling)') # Set a descriptive title for this plot
15 plt.axis('off') # Hide axis ticks and coordinates
16
17 # Create the right subplot (1 row, 2 columns, 2nd position)
18 plt.subplot(1, 2, 2) # Position this image within a grid layout
19 plt.imshow(cv2.cvtColor(pixelated, cv2.COLOR_BGR2RGB)) # Display the blurred/blocky low-sampling image
20 plt.title('Pixelated (Low Sampling)') # Set a descriptive title for this plot
21 plt.axis('off') # Hide axis ticks and coordinates
22 plt.show() # Render both subplots to the screen
```

Low-Res Shape: (25, 50, 3)

Original (High Sampling)



Pixelated (Low Sampling)



```

1  # Define a list of different resolutions to test (Low, Medium, High)
2  sizes = [(50, 25), (200, 100), (400, 200)]      # Width x Height tuples
3  # Labels for our plot titles
4  labels = ['Low (50x25)', 'Medium (200x100)', 'High (400x200)']      # Define corresponding text labels
5
6  plt.figure(figsize=(14, 4))                      # Create a wide figure window
7
8  # Loop through each size and label simultaneously
9  for i, (size, label) in enumerate(zip(sizes, labels)):      # Iterate through the parameters
10     # Step 1: Downsample the original image to the specific 'size' to simulate the sensor resolution
11     small = cv2.resize(img, size, interpolation=cv2.INTER_AREA) # INTER_AREA is best for shrinking images
12
13     # Step 2: Upsample it back to 400x200 so we can compare them all at the same physical size
14     restored = cv2.resize(small, (400, 200), interpolation=cv2.INTER_NEAREST) # Upsample image
15
16     plt.subplot(1, 3, i + 1)                      # Create a subplot (1 row, 3 columns, position i+1)
17     plt.imshow(cv2.cvtColor(restored, cv2.COLOR_BGR2RGB)) # Show the image
18     plt.title(label)                              # Set the title to the current sampling rate label
19     plt.axis('off')                               # Hide the axis
20
21 plt.suptitle('Effect of Different Sampling Rates') # Add a main title for the whole window
22 plt.tight_layout()                               # Automatically adjust spacing between plots
23 plt.show()                                       # Display the final comparison

```

Effect of Different Sampling Rates

Low (50x25)



Medium (200x100)



High (400x200)

